

1. Java Exceptions

An exception is a problem that arises during the execution of a program. An exception can occur for many different reasons, including the following:

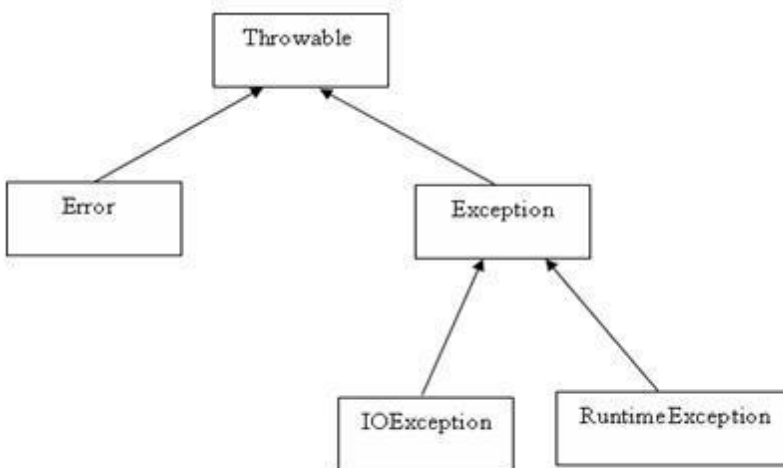
- A user has entered invalid data.
- A file that needs to be opened cannot be found.
- A network connection has been lost in the middle of communications or the JVM has run out of memory.

Some of these exceptions are caused by user error, others by programmer error, and others by physical resources that have failed in some manner. To understand how exception handling works in Java, we need to understand the three categories of exceptions:

- **Checked exceptions:** A checked exception is an exception that is typically a user error or a problem that cannot be foreseen by the programmer. For example, if a file is to be opened, but the file cannot be found, an exception occurs. These exceptions cannot simply be ignored at the time of compilation.
- **Runtime exceptions:** A runtime exception is an exception that occurs but that probably could have been avoided by the programmer. As opposed to checked exceptions, runtime exceptions are ignored at the time of compilation.
- **Errors:** These are not exceptions at all, but are the problems that arise beyond the control of the user or the programmer. Errors are typically ignored in our code because we can rarely do anything about an error. For example, if a stack overflow occurs, an error will arise. They are also ignored at the time of compilation.

2. Exception Hierarchy:

All exception classes are subtypes of the `java.lang.Exception` class. The exception class is a subclass of the `Throwable` class. Other than the exception class there is another subclass called `Error` which is derived from the `Throwable` class. Errors are not normally trapped from the Java programs. These conditions normally happen in case of severe failures, which are not handled by the java programs. Errors are generated to indicate errors generated by the runtime environment. Example : JVM is out of Memory. Normally programs cannot recover from errors. The `Exception` class has two main subclasses: `IOException` class and `RuntimeException` Class.



Lecture Notes By Rashmi Paneru Pandey

Java's Built-in Exceptions

Java defines several exception classes inside the standard package `java.lang`. The most general of these exceptions are subclasses of the standard type `RuntimeException`. Since `java.lang` is implicitly imported into all Java programs, most exceptions derived from `RuntimeException` are automatically available. Java defines several other types of exceptions that relate to its various class libraries. Following is the list of Java Unchecked `RuntimeException`. Java's Unchecked `RuntimeException` subclasses defined in `java.lang` are:

Exception	Meaning
<code>ArithmeticException</code>	Arithmetic error, such as divide-by-zero.
<code>ArrayIndexOutOfBoundsException</code>	Array index is out-of-bounds.
<code>ArrayStoreException</code>	Assignment to an array element of an incompatible type.
<code>ClassCastException</code>	Invalid cast.
<code>EnumConstantNotPresentException</code>	An attempt is made to use an undefined enumeration value.
<code>IllegalArgumentException</code>	Illegal argument used to invoke a method.
<code>IllegalMonitorStateException</code>	Illegal monitor operation, such as waiting on an unlocked thread.
<code>IllegalStateException</code>	Environment or application is in incorrect state.
<code>IllegalThreadStateException</code>	Requested operation not compatible with current thread state.
<code>IndexOutOfBoundsException</code>	Some type of index is out-of-bounds.
<code>NegativeArraySizeException</code>	Array created with a negative size.
<code>NullPointerException</code>	Invalid use of a null reference.
<code>NumberFormatException</code>	Invalid conversion of a string to a numeric format.
<code>SecurityException</code>	Attempt to violate security.
<code>StringIndexOutOfBoundsException</code>	Attempt to index outside the bounds of a string.
<code>TypeNotPresentException</code>	Type not found.
<code>UnsupportedOperationException</code>	An unsupported operation was encountered.

Java's Checked Exceptions Defined in `java.lang` are:

Exception	Meaning
<code>ClassNotFoundException</code>	Class not found.
<code>CloneNotSupportedException</code>	Attempt to clone an object that does not implement the <code>Cloneable</code> interface.
<code>IllegalAccessException</code>	Access to a class is denied.
<code>InstantiationException</code>	Attempt to create an object of an abstract class or interface.
<code>InterruptedException</code>	One thread has been interrupted by another thread.
<code>NoSuchFieldException</code>	A requested field does not exist.
<code>NoSuchMethodException</code>	A requested method does not exist.

3. Catching Exceptions:

A method catches an exception using a combination of the try and catch keywords. A try/catch block is placed around the code that might generate an exception. Code within a try/catch block is referred to as protected code, and the syntax for using try/catch looks like the following:

```
try {  
    //Protected code  
}  
catch(ExceptionName e1) {  
    //Catch block  
}
```

A catch statement involves declaring the type of exception we are trying to catch. If an exception occurs in protected code, the catch block (or blocks) that follows the try is checked. If the type of exception that occurred is listed in a catch block, the exception is passed to the catch block much as an argument is passed into a method parameter.

Example:

The following is an array is declared with 2 elements. So, when the code tries to access the 3rd element of the array it throws an exception.

```
import java.io.*;  
public class ExcepTest  
{  
    public static void main(String args[])  
    {  
        try  
        {  
            int a[]=new int[2];  
            System.out.println("Access element three :"+ a[3]);  
        }  
        catch(ArrayIndexOutOfBoundsException e)  
        {  
            System.out.println("Exception thrown :"+ e);  
        }  
        System.out.println("Out of the block");  
    }  
}
```

Output:

```
Exception thrown :java.lang.ArrayIndexOutOfBoundsException:3  
Out of the block
```

➤ **Multiple catch Blocks:**

A try block can be followed by multiple catch blocks. The syntax for multiple catch blocks looks like the following:

```
try  
{  
    //Protected code  
}  
catch(ExceptionType1 e1)
```

```
{
//Catch block
}
catch(ExceptionType2 e2)
{
//Catch block
}
catch(ExceptionType3 e3)
{
//Catch block
}
```

Example

```
class MultiCatch {
public static void main(String args[]) {
try {
int a = args.length;
System.out.println("a = " + a);
int b = 42 / a;
int c[] = { 1 };
c[42] = 99;
} catch(ArithmeticException e) {
System.out.println("Divide by 0: " + e);
} catch(ArrayIndexOutOfBoundsException e) {
System.out.println("Array index oob: " + e);
}
System.out.println("After try/catch blocks.");
}
}
```

Output:

```
a = 0
Divide by 0: java.lang.ArithmeticException: / by zero
After try/catch blocks.
a = 1
Array index oob: java.lang.ArrayIndexOutOfBoundsException:42
After try/catch blocks.
```

4. The throws/throw Keywords:

The throw clause can be used to throw the exception explicitly. The general form of **throw** is shown here:

```
throw ThrowableInstance;
```

Here, *ThrowableInstance* must be an object of type **Throwable** or a subclass of **Throwable**.

We can throw an exception, either a newly instantiated one or an exception that we just caught, by using the **throw** keyword.

Demonstrate throw

```
class ThrowDemo {
    static void demoproc() {
        try {
            throw new NullPointerException("demo");
        }
        catch(NullPointerException e) {
            System.out.println("Caught inside demoproc.");
            throw e; // rethrow the exception
        }
    }
    public static void main(String args[]) {
        try {
            demoproc();
        }
        catch(NullPointerException e) {
            System.out.println("Recaught: " + e);
        }
    }
}
```

Output:

Caught inside demoproc.

Recaught: java.lang.NullPointerException

If a method does not handle a checked exception, the method must declare it using the **throws** keyword. The throws keyword appears at the end of a method's signature. A **throws** clause lists the types of exceptions that a method might throw.

The general form of a method declaration that includes a **throws** clause:

```
type method-name(parameter-list) throws exception-list
{
    // body of method
}
```

Example showing the throws clause:

```
class ThrowsDemo {
    static void throwOne() throws IllegalAccessException {
        System.out.println("Inside throwOne.");
        throw new IllegalAccessException("demo");
    }
    public static void main(String args[]) {
        try {
            throwOne();
        } catch (IllegalAccessException e) {
            System.out.println("Caught " + e);
        }
    }
}
```

Output:

Inside throwOne
caught java.lang.IllegalAccessException: demo

5. The finally Keyword

The finally keyword is used to create a block of code that follows a try block. A finally block of code always executes, whether or not an exception has occurred. **finally** creates a block of code that will be executed after a **try/catch** block has completed. If an exception is thrown, the **finally** block will execute even if no **catch** statement matches the exception. Any time a method is about to return to the caller from inside a **try/catch** block, the **finally** clause is also executed just before the method returns. The **finally** clause is optional. However, each **try** statement requires at least one **catch** or a **finally** clause.

A finally block appears at the end of the catch blocks and has the following syntax:

```
try
{
    //Protected code
}
catch(ExceptionType1 e1)
{
    //Catch block
}
catch(ExceptionType2 e2)
{
    //Catch block
}
catch(ExceptionType3 e3)
{
    //Catch block
}
finally
{
    //The finally block always executes.
}
```

Example:

```
class FinallyDemo {
    // Through an exception out of the method.
    static void procA() {
        try {
            System.out.println("inside procA");
            throw new RuntimeException("demo");
        } finally {
            System.out.println("procA's finally");
        }
    }
    // Return from within a try block.
```

```
static void procB() {
try {
System.out.println("inside procB");
return;
} finally {
System.out.println("procB's finally");
}
}
// Execute a try block normally.
static void procC() {
try {
System.out.println("inside procC");
} finally {
System.out.println("procC's finally");
}
}
public static void main(String args[]) {
try {
procA();
} catch (Exception e) {
System.out.println("Exception caught");
}
procB();
procC();
}
}
```

Output

```
inside procA
procA's finally
Exception caught
inside procB
procB's finally
inside procC
procC's finally
```

6. Declaring our own Exception:

We can create our own exceptions in Java. We just need to extend the Exception class to create our own Exception class. We should consider the following points to write our own exception classes:

- All exceptions must be a child of Throwable class.
- If we want to write a checked exception that is automatically enforced by the Handle or Declare Rule, we need to extend the Exception class.
- If we want to write a runtime exception, we need to extend the RuntimeException class.

Lecture Notes By Rashmi Paneru Pandey

We can define our own Exception class as below:

```
class MyException extends Exception
{
```

```
}
```

Example:

// This program creates a custom exception type.

```
import java.lang.Exception;
class MyException extends Exception {
    private int detail;
    MyException(int a) {
        detail = a;
    }
    public String toString() {
        return "MyException[" + detail + "]";
    }
}
class ExceptionDemo {
    static void compute(int a) throws MyException {
        System.out.println("Called compute(" + a + ")");
        if(a > 10)
            throw new MyException(a);
        System.out.println("Normal exit");
    }
    public static void main(String args[]) {
        try {
            compute(1);
            compute(20);
        } catch (MyException e) {
            System.out.println("Caught " + e);
        }
    }
}
```

Output

```
Called compute(1)
Normal exit
Called compute(20)
Caught MyException[20]
```

This example defines a subclass of **Exception** called **MyException**. This subclass has only a constructor and an overloaded **toString()** method that displays the value of the exception. The **ExceptionDemo** class defines a method named **compute()** that throws a **MyException** object. The exception is thrown when **compute()**'s integer parameter is greater than 10. The **main()** method sets up an exception handler for **MyException**, then calls **compute()** with a legal value (less than 10) and an illegal one to show both paths through the code.

7. Tips On Handling Exception

➤ **Use a single,system wide exception class:**

Instead of creating separate classes for each exception type, create just one. And make it extend RuntimeException. This will reduce our class count and remove the need to declare exceptions we aren't going to handle anyway.

➤ **Add dynamic field to our exception:**

Good exception handling means also recording relevant data, not just the stack trace. Doing this will save our big time when trying to diagnose and reproduce errors. The easiest way to do this is to add a java.util.Map field to the exception.

➤ **Close or release resource in finally block**

This is a well-known best practice in Java and quite a standard, while dealing with networking and IO classes. Closing resources in finally block guarantees that precious and scarce resource released properly in case of normal and aborted execution, guaranteed by finally block.

➤ **Always provide meaning full message on Exception**

Message of Exception is the most important place, where we can point out cause of problem because this is the first place every programmer looks upon. Always try to provide precise and factual information here. For example, compare two Exception messages for IllegalArgumentException :

message 1: "Incorrect argument for method"

message 2: "Illegal value for \${argument}: \${value}"

First one just says that argument is illegal or incorrect, but second one include both name of argument and its illegal value which is important to point out cause of error.

➤ **Use Standard Exceptions**

Using standard Exception instead of creating own Exception is much better in terms of maintenance and consistency. Reusing standard exception makes code more readable, because most of Java developers are familiar with standard RuntimeException from JDK like, IllegalStateException, IllegalArgumentException or NullPointerException, and they will immediately be able to know purpose of Exception, instead of looking out another place on code or docs to find out purpose of user defined Exceptions.

➤ **Document Exception thrown by any method**

Java provides throw and throws keyword to throw exception and in javadoc we have @throw to document possible Exception thrown by any method. With proper documentation of Exception thrown by any method we can potentially alert anyone who is using it.

8. Debugging

Debugging is the process to locate the error and fix it. It is the process of removing errors from programs. Testing finds errors and debugging localizes and repairs them. Programmers learn a variety of debugging techniques that involve actually executing the code on a computer.

➤ **Rudimentary Technique: Print Statements**

A rudimentary debugging technique is debugging through the addition of print statements to a program. These statements are strategically placed to show the flow of control and the values of

Lecture Notes By Rashmi Paneru Pandey

key variables. The output produced may make the problem obvious or be used to successively narrow down the problem location.

Although inserting print statements is a simple technique that can be used on virtually any system, it has the major disadvantage that the programmer has to specify what to display before executing the program. If one realizes that key information is not displayed, the program usually has to be edited to add additional print statements and then recompiled and restarted.

While debugging using print statements is usually not the best technique for finding bugs, it may be useful in a context with limited interactivity or language support. If debugging using print statements is done, many programmers like to be able to control the output without having to modify each print statement.

➤ **Advanced Technique: Using an Interactive Debugger**

Good development environments provide a tool known as an interactive debugger, or more simply, a debugger. Debuggers allow examination of the status of a program that is currently executing. This provides one with a "window" into the operation of a program, through which control flow and variable values are visible whenever sought.

The Java tools i.e. the Java Development Kit, include a debugger called jdb. It allows us to do many useful things, such as examine variable values, see which methods are currently executing and who invoked them, and set breakpoints to make the program stop at various points.

- When compiling with javac, use the -g flag to include debug information: For example, javac -g DoubleList.java
- **jdb DoubleList**
Starts jdb and loads the class DoubleList. We can get the same thing by starting jdb with no arguments and then using the load command to load the class DoubleList manually;
- Tell jdb to stop every time a certain method is entered and exited:
stop in DoubleList.main
This is a *breakpoint*; as jdb runs the program for us, it will stop at any breakpoint we have set. We can set several different breakpoints. We may stop at DoubleList:88 to set a breakpoint at a particular line of DoubleList.java.
- **list**
It will list the source code and show which line is about to be executed next.
- **next**
It executes one line of code, and then stops again.
- **cont**
It continue running the program until the next breakpoint is hit.
- **where**
It list the current execution stack, that is find out not only where we are, but why. The stack shows not only which method is currently running, but where this method was called from, and so forth.

9. **Stream**

The java.io package contains nearly every class needed to perform input and output (I/O) in Java. All these streams represent an input source and an output destination. The stream in the java.io package supports many data such as primitives, Object, localized characters, etc. A stream is an abstraction that either produces or consumes information. A stream is linked to a physical device by the Java I/O system. All streams behave in the same manner, even if the actual physical

Lecture Notes By Rashmi Paneru Pandey

devices to which they are linked differ. Thus, the same I/O classes and methods can be applied to any type of device. A stream can be defined as a sequence of data. The `InputStream` is used to read data from a source and the `OutputStream` is used for writing data to a destination.

➤ Byte Streams

Java byte streams are used to perform input and output of 8-bit bytes. Byte streams are defined by using two class hierarchies. At the top are two abstract classes: **`InputStream`** and **`OutputStream`**. Each of these abstract classes has several concrete subclasses that handle the differences between various devices, such as disk files, network connections, and even memory buffers. Though there are many classes related to byte streams but the most frequently used classes are **`FileInputStream`** and **`FileOutputStream`**. The abstract classes **`InputStream`** and **`OutputStream`** define several key methods that the other stream classes implement. Two of the most important are **`read()`** and **`write()`**, which, respectively, read and write bytes of data. Both methods are declared as abstract inside **`InputStream`** and **`OutputStream`**. Following is an example which makes use of these two classes to copy an input file into an output file:

```
import java.io.*;
public class CopyFile {
    public static void main(String args[]) throws IOException
    {
        FileInputStream in = null;
        FileOutputStream out = null;
        try {
            in = new FileInputStream("input.txt");
            out = new FileOutputStream("output.txt");
            int c;
            while ((c = in.read()) != -1) {
                out.write(c);
            }
        } finally {
            if (in != null) {
                in.close();
            }
            if (out != null) {
                out.close();
            }
        }
    }
}
```

The Byte Stream Classes

Stream Class

`BufferedInputStream`

`BufferedOutputStream`

`ByteArrayInputStream`

Meaning

Buffered input stream

Buffered output stream

Input stream that reads from a byte array

Lecture Notes By Rashmi Paneru Pandey

ByteArrayOutputStream	Output stream that writes to a byte array
DataInputStream	An input stream that contains methods for reading the Java standard data types
DataOutputStream	An output stream that contains methods for writing the Java standard data types
FileInputStream	Input stream that reads from a file
FileOutputStream	Output stream that writes to a file
FilterInputStream	Implements InputStream
FilterOutputStream	Implements OutputStream
InputStream	Abstract class that describes stream input
ObjectInputStream	Input stream for objects
ObjectOutputStream	Output stream for objects
OutputStream	Abstract class that describes stream output
PipedInputStream	Input pipe
PipedOutputStream	Output pipe
PrintStream	Output stream that contains print() and println()
PushbackInputStream	Input stream that supports one-byte “unget,” which returns a byte to the input stream
RandomAccessFile	Supports random access file I/O
SequenceInputStream	Input stream that is a combination of two or more input streams that will be read sequentially, one after the other

➤ Character Streams

Java **Byte** streams are used to perform input and output of 8-bit bytes, where as Java **Character** streams are used to perform input and output for 16-bit unicode. Character streams are defined by using two class hierarchies. At the top are two abstract classes, **Reader** and **Writer**. Though there are many classes related to character streams but the most frequently used classes are **FileReader** and **FileWriter**.. Though internally FileReader uses FileInputStream and FileWriter uses FileOutputStream but here major difference is that FileReader reads two bytes at a time and FileWriter writes two bytes at a time. The abstract classes **Reader** and **Writer** define several key methods that the other stream classes implement. Two of the most important methods are **read()** and **write()**, which read and write characters of data, respectively. We can re-write above example which makes use of these two classes to copy an input file (having unicode characters) into an output file:

```
import java.io.*;
public class CopyFile {
    public static void main(String args[]) throws IOException
    {
        FileReader in = null;
        FileWriter out = null;
        try {
            in = new FileReader("input.txt");
```

Lecture Notes By Rashmi Paneru Pandey

```
out = new FileWriter("output.txt");
int c;
while ((c = in.read()) != -1) {
    out.write(c);
}
}finally {
    if (in != null) {
        in.close();
    }
    if (out != null) {
        out.close();
    }
}
}
```

The Character Stream I/O Classes

Stream Class

BufferedReader
BufferedWriter
CharArrayReader
CharArrayWriter
FileReader
FileWriter
FilterReader
FilterWriter
InputStreamReader
LineNumberReader
OutputStreamWriter
PipedReader
PipedWriter
PrintWriter
PushbackReader

Reader
StringReader
StringWriter
Writer

Meaning

Buffered input character stream
Buffered output character stream
Input stream that reads from a character array
Output stream that writes to a character array
Input stream that reads from a file
Output stream that writes to a file
Filtered reader
Filtered writer
Input stream that translates bytes to characters
Input stream that counts lines
Output stream that translates characters to bytes
Input pipe
Output pipe
Output stream that contains print() and println()
Input stream that allows characters to be returned to the input stream
Abstract class that describes character stream input
Input stream that reads from a string
Output stream that writes to a string
Abstract class that describes character stream output